



**UNIVERSIDAD TECNOLÓGICA DE PUEBLA
INGENIERÍA EN DESARROLLO DE
SOFTWARE MULTIPLATAFORMA**

**APLICACIÓN ORIENTADA
A SERVICIOS**

REPORTE — SUBPRODUCTO NO. 6

**ASIGNATURA: APLICACIONES WEB
PROGRESIVAS**

**DOCENTE: MATHER XÓCHITL MENDOZA
PÍSCIL**

**ALUMNO: PÉREZ ALCÁNTARA ALEJANDRO
10° “D”**

FECHA: 28 DE SEPTIEMBRE DE 2026

ÍNDICE

1. Introducción.....	3
2. ¿Qué es una Aplicación Orientada a Servicios?	3
3. Ilustración: ¿Cómo funciona una Aplicación Orientada a Servicios?	4
4. Características Principales	6
5. Ventajas y Desventajas	6
6. Comparativa: Orientada a Servicios vs. Móvil vs. Web vs. Escritorio vs. Multiplataforma	7
7. Tecnología Utilizada	8
8. Ejemplo de Funcionamiento: HabitPulse.....	9
9. Tabla Comparativa: Requisitos de una Aplicación Orientada a Servicios vs. HabitPulse	15
10. Lista de Cotejo.....	16
11. Conclusión	17
12. URL de la Aplicación	17
Referencias.....	18

1. Introducción

A lo largo de esta materia he documentado los distintos enfoques de desarrollo de software: en el Subproducto No. 1 documenté el desarrollo nativo, usando como ejemplo propio mi proyecto de reloj inteligente en Wear OS; en el Subproducto No. 2 documenté el desarrollo web, usando como ejemplo propio MediCore Professional System; en el Subproducto No. 3 documenté el desarrollo de escritorio, usando como ejemplo propio TaskDesk; en el Subproducto No. 4 documenté el desarrollo móvil, usando como ejemplo propio PediatriTrack Global; y en el Subproducto No. 5 documenté el desarrollo multiplataforma, usando como ejemplo propio ExpenseTrack Lite. En este sexto reporte cierro el panorama con el enfoque de aplicaciones orientadas a servicios: sistemas en los que la lógica y los datos se exponen a través de un servicio independiente (una API), que distintos clientes —web, móvil u otros servicios— pueden consumir por red usando un contrato fijo, sin conocer ni depender de cómo está implementado ese servicio por dentro.

A lo largo del documento defino el concepto de aplicación orientada a servicios, ilustro su funcionamiento mediante un diagrama, listo sus características principales, sus ventajas y desventajas, la comparo frente a los otros enfoques que ya documenté en esta materia (móvil, web, escritorio, multiplataforma), documento la tecnología utilizada y, como evidencia propia, presento HabitPulse: un servicio REST construido con Node.js y Express para el seguimiento de hábitos diarios, junto con un cliente web que lo consume exclusivamente a través de peticiones HTTP en formato JSON. Cierro el documento con una tabla comparativa entre lo que exige la definición de una aplicación orientada a servicios y lo que realmente implementé, una lista de cotejo de autoevaluación y la URL del repositorio donde está el código.

2. ¿Qué es una Aplicación Orientada a Servicios?

Una aplicación orientada a servicios es un sistema de software cuya funcionalidad se organiza en uno o varios servicios independientes, cada uno con una responsabilidad concreta, que se comunican entre sí y con sus clientes a través de un contrato bien definido —típicamente peticiones HTTP con datos en formato JSON o XML— en lugar de compartir memoria o código directamente. La idea central es separar por completo al que ofrece la funcionalidad (el servicio o la API) del que la consume (el cliente): el cliente no necesita saber en qué lenguaje está escrito el servicio, cómo guarda sus datos o dónde está desplegado; solo necesita conocer el contrato: qué rutas existen, qué datos hay que enviar y qué datos va a recibir de vuelta.

Esto conecta con lo que ya documenté en los subproductos anteriores: en el Subproducto No. 2 (MediCore) ya usaba peticiones a un backend para leer y guardar información, y en el Subproducto No. 4 (PediatriTrack Global) la app consumía servicios externos como Firebase y openFDA. En este reporte hago explícito y central ese patrón: en vez de que un backend sea un detalle de implementación de una sola app, HabitPulse existe como un servicio HTTP autónomo, publicado en su propio puerto, que cualquier cliente —el sitio web incluido en este proyecto, una futura app móvil, u otro programa que le haga peticiones con curl— puede consumir de la misma forma, sin tocar su código interno.

2.1 Tipos de Enfoques Orientados a Servicios

Identifique tres formas principales en las que se organiza una aplicación orientada a servicios:

- **API REST sobre HTTP/JSON:** el servicio expone rutas (endpoints) que representan recursos (por ejemplo /api/habitos) y usa los verbos HTTP (GET, POST, PUT, DELETE) para operar sobre ellos; es el enfoque que uso en HabitPulse.
- **Microservicios:** la aplicación se divide en varios servicios pequeños e independientes (usuarios, pagos, notificaciones, etc.), cada uno con su propia base de datos y ciclo de despliegue, que se comunican entre sí por red.
- **SOA clásica con middleware de mensajería (SOAP, colas de mensajes, ESB):** los servicios se comunican a través de un bus o intermediario central que enruta, transforma y coordina los mensajes entre ellos, típico de sistemas empresariales grandes y más antiguos que las APIs REST modernas.

3. Ilustración: ¿Cómo funciona una Aplicación Orientada a Servicios?

El siguiente diagrama ilustra la arquitectura de HabitPulse: varios clientes independientes (el cliente web incluido en el proyecto, un futuro cliente móvil, u otro servicio o script) le hacen peticiones HTTP al mismo servicio, que expone un contrato fijo de endpoints REST. El servicio, a su vez, delega el almacenamiento de los datos a su propia capa interna, que podría cambiarse sin que ningún cliente se entere, siempre que el contrato HTTP/JSON se mantenga igual.

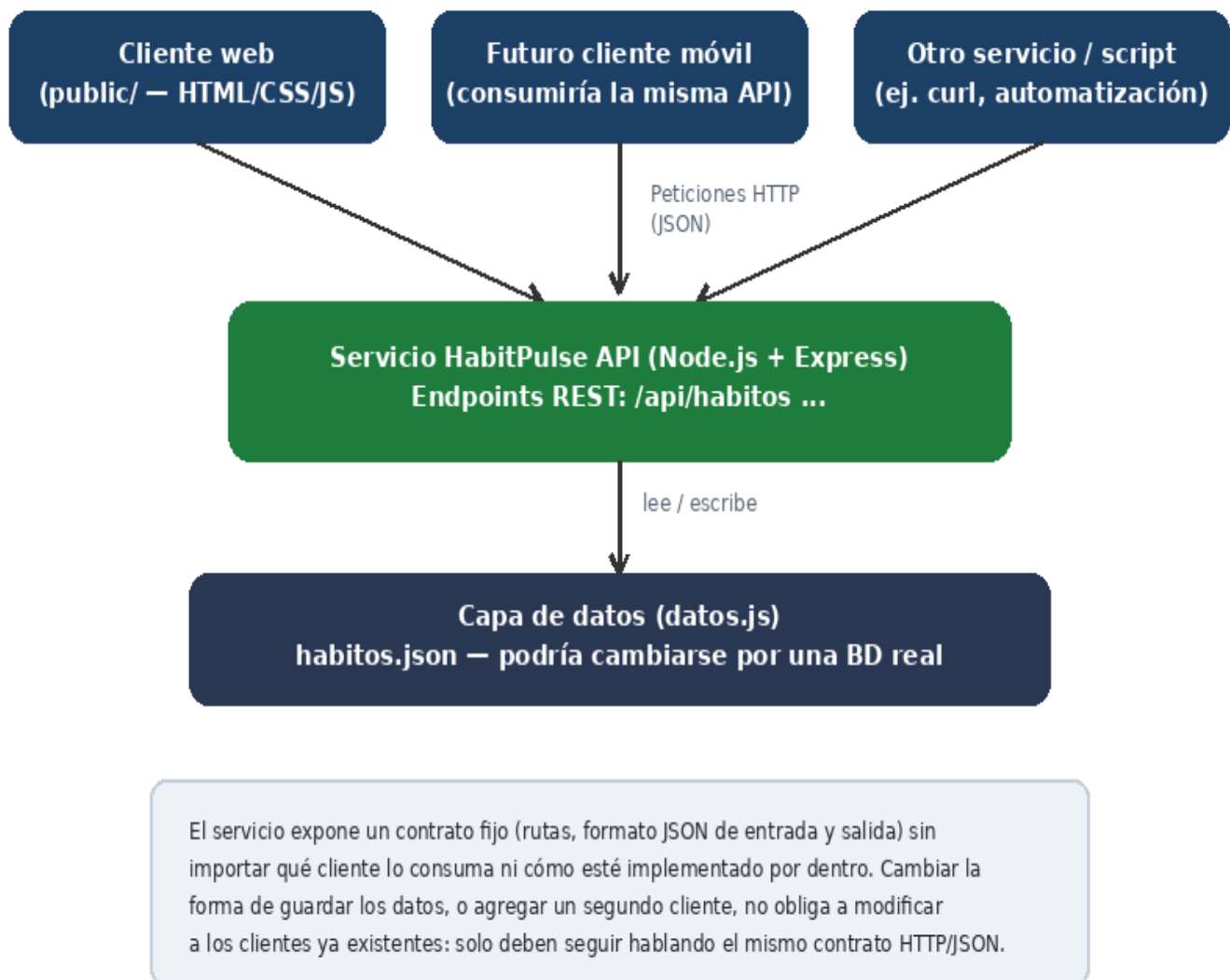


Figura 1. Arquitectura orientada a servicios de HabitPulse: varios clientes independientes consumen un único servicio HTTP/JSON, que a su vez delega el almacenamiento a su propia capa de datos.

4. Características Principales

- **Separación entre servicio y cliente:** la lógica de negocio y los datos viven en el servicio; el cliente solo consume el contrato HTTP, sin conocer la implementación interna.
- **Contrato explícito y estable:** las rutas, los métodos HTTP y el formato de los datos (JSON) están definidos de antemano, de modo que distintos clientes pueden consumir el mismo servicio sin coordinarse entre sí.
- **Comunicación por red, sin estado compartido en memoria:** cada petición HTTP es independiente (stateless); el servicio no depende de que el cliente recuerde nada entre una petición y otra, más allá de los datos que el propio servicio le devuelve.
- **Reusabilidad entre múltiples clientes:** el mismo servicio puede ser consumido por un sitio web, una app móvil, otro servicio backend o incluso un script de línea de comandos, sin duplicar la lógica de negocio en cada uno.
- **Escalabilidad y despliegue independientes:** el servicio puede desplegarse, escalarse o reiniciarse por separado de sus clientes, ya que la única dependencia entre ambos es la red y el contrato HTTP.
- **Mayor complejidad operativa:** depende de que la red esté disponible, y requiere manejar explícitamente errores de conexión, tiempos de espera, formatos de error y, en muchos casos, autenticación entre el cliente y el servicio.

5. Ventajas y Desventajas

A partir de las características anteriores, resumo las principales ventajas y desventajas del desarrollo orientado a servicios, tal como hice en los subproductos anteriores para los otros enfoques:

VENTAJAS	DESVENTAJAS
Un mismo servicio puede reutilizarse desde varios clientes distintos (web, móvil, otros servicios) sin duplicar lógica.	Depende de la disponibilidad de la red; si el servicio o la conexión fallan, el cliente pierde funcionalidad.
El servicio y los clientes se pueden desarrollar, desplegar y escalar de forma independiente entre sí.	Agrega complejidad: hay que definir y mantener un contrato (rutas, formatos, códigos de error) entre servicio y clientes.
Facilita cambiar la implementación interna del servicio (por ejemplo, la base de datos) sin tocar a los clientes.	Cada petición viaja por red, lo que agrega latencia comparado con llamar una función local dentro del mismo proceso.
Permite centralizar reglas de negocio y datos en un solo lugar, evitando inconsistencias entre distintas apps.	Requiere manejar explícitamente seguridad (autenticación, CORS, validación) porque el servicio queda expuesto en la red.

6. Comparativa: Orientada a Servicios vs. Móvil vs. Web vs. Escritorio vs. Multiplataforma

Para cerrar el panorama de los enfoques que documenté a lo largo de esta materia, comparo aquí el desarrollo orientado a servicios frente a los otros cuatro que ya presenté: móvil (Subproducto No. 4, PediatriTrack Global), web (Subproducto No. 2, MediCore), escritorio nativo (Subproducto No. 3, TaskDesk) y multiplataforma (Subproducto No. 5, ExpenseTrack Lite):

CRITERIO	ORIENTADA A SERVICIOS	MÓVIL	WEB	ESCRITORIO	MULTIPLATAFORMA
Lenguaje	Node.js/Express (servicio) + HTML/CSS/JS (cliente)	JavaScript/TypeScript (React Native), Kotlin/Swift	HTML, CSS, JS (+ backend)	Python, C#, C++, Java	HTML, CSS, JS + Node.js (Electron)
Rendimiento	Depende de la red y del propio servicio	Alto	Medio-bajo	Alto	Medio (runtime embebido)
Acceso a recursos	Solo lo que el contrato del servicio expone	Amplio (cámara, GPS, sensores)	Limitado (APIs del navegador)	Amplio (archivos, periféricos)	Amplio (archivos, notificaciones)
Instalación	No para el cliente web; el servicio se despliega aparte	Sí, desde tienda de apps	No requiere, se accede por URL	Sí (o requiere runtime)	Sí, un instalador por SO
Funciona sin internet	No (depende de la red para hablar con el servicio)	Sí, parcialmente	Muy limitado	Sí	Sí
Costo de desarrollo	Medio (servicio + cliente(s) por separado)	Medio (React Native: 1 código, 2 plataformas)	Bajo (1 solo código)	Medio-alto (1 código por SO)	Bajo-medio (1 código, 3 SO)
Ejemplos	HabitPulse, APIs de Stripe/Firebase, microservicios	PediatriTrack Global, WhatsApp	MediCore, Google Docs, PWA	TaskDesk, mi app Wear OS	ExpenseTrack Lite, VS Code

A diferencia de los otros cuatro enfoques, que describen principalmente cómo se construye y distribuye un cliente, el enfoque orientado a servicios describe cómo se organiza la lógica detrás de esos clientes: de hecho, cualquiera de las apps web, móviles, de escritorio o multiplataforma que ya documenté podría construirse consumiendo servicios independientes como HabitPulse, en vez de tener toda su lógica embebida en el propio cliente. Elegí Node.js y Express para HabitPulse precisamente porque ya había usado JavaScript en el resto de la materia, y porque es una de las combinaciones más comunes para construir APIs REST ligeras y bien documentadas.

7. Tecnología Utilizada

Para HabitPulse utilicé la siguiente tecnología:

COMPONENTE	TECNOLOGÍA	FUNCIÓN
Servicio / API REST	Node.js 22 + Express 4	Expone los endpoints HTTP /api/habitos para crear, listar, completar y eliminar hábitos
Formato de intercambio	JSON	Formato en el que el servicio recibe y devuelve todos los datos, sin importar quién lo consuma
Capa de datos del servicio	Módulo datos.js + archivo habits.json	Aísla cómo se guardan los datos; podría sustituirse por una base de datos real sin afectar a los clientes
CORS	Paquete cors	Permite que clientes en otros orígenes (otro dominio o puerto) consuman el servicio por HTTP
Cliente web (evidencia 1)	HTML, CSS y JavaScript (fetch)	Interfaz que consume el servicio exclusivamente por peticiones HTTP, mostrando además cada petición y respuesta real
Cliente de línea de comandos (evidencia 2)	curl	Demuestra que el servicio puede consumirse fuera del cliente web, con cualquier programa que hable HTTP/JSON
Control de versiones	Git y GitHub	Repositorio del proyecto con el historial de cambios del código del servicio y del cliente web

Elegí Express sobre Node.js porque me permite declarar cada endpoint de forma explícita (método HTTP + ruta) y separar con claridad las tres capas del servicio: las rutas (server/index.js), que reciben las peticiones y arman las respuestas; la capa de datos (server/datos.js), que es la única parte del proyecto que sabe que los hábitos se guardan en un archivo JSON; y el cliente web (carpeta public/), que no tiene ninguna referencia directa a esos archivos y solo conoce las rutas HTTP publicadas. Esa separación es la que permite, por ejemplo, demostrar que el mismo servicio responde igual tanto si lo llama el cliente web como si lo llamo directamente con curl desde la terminal, sin cambiar una sola línea del servidor.

8. Ejemplo de Funcionamiento: HabitPulse

Como ejemplo propio de aplicación orientada a servicios, presento HabitPulse: un servicio REST para el seguimiento de hábitos diarios, que permite crear un hábito, marcarlo como completado el día de hoy, eliminarlo, y consultar su racha de días consecutivos y su historial. El servicio no incluye ninguna lógica de interfaz: solo expone endpoints HTTP que reciben y devuelven JSON.

Construí, además del servicio, un cliente web (carpeta public/) que lo consume exclusivamente por fetch, y agregué un panel dentro del propio cliente que muestra en tiempo real la última petición HTTP realizada y la respuesta JSON recibida, como evidencia de que toda la comunicación pasa por el contrato del servicio. A continuación documento con capturas reales tanto el cliente web como el servicio consumido directamente desde la terminal con curl, para dejar claro que el servicio funciona de manera independiente de cualquier cliente en particular.

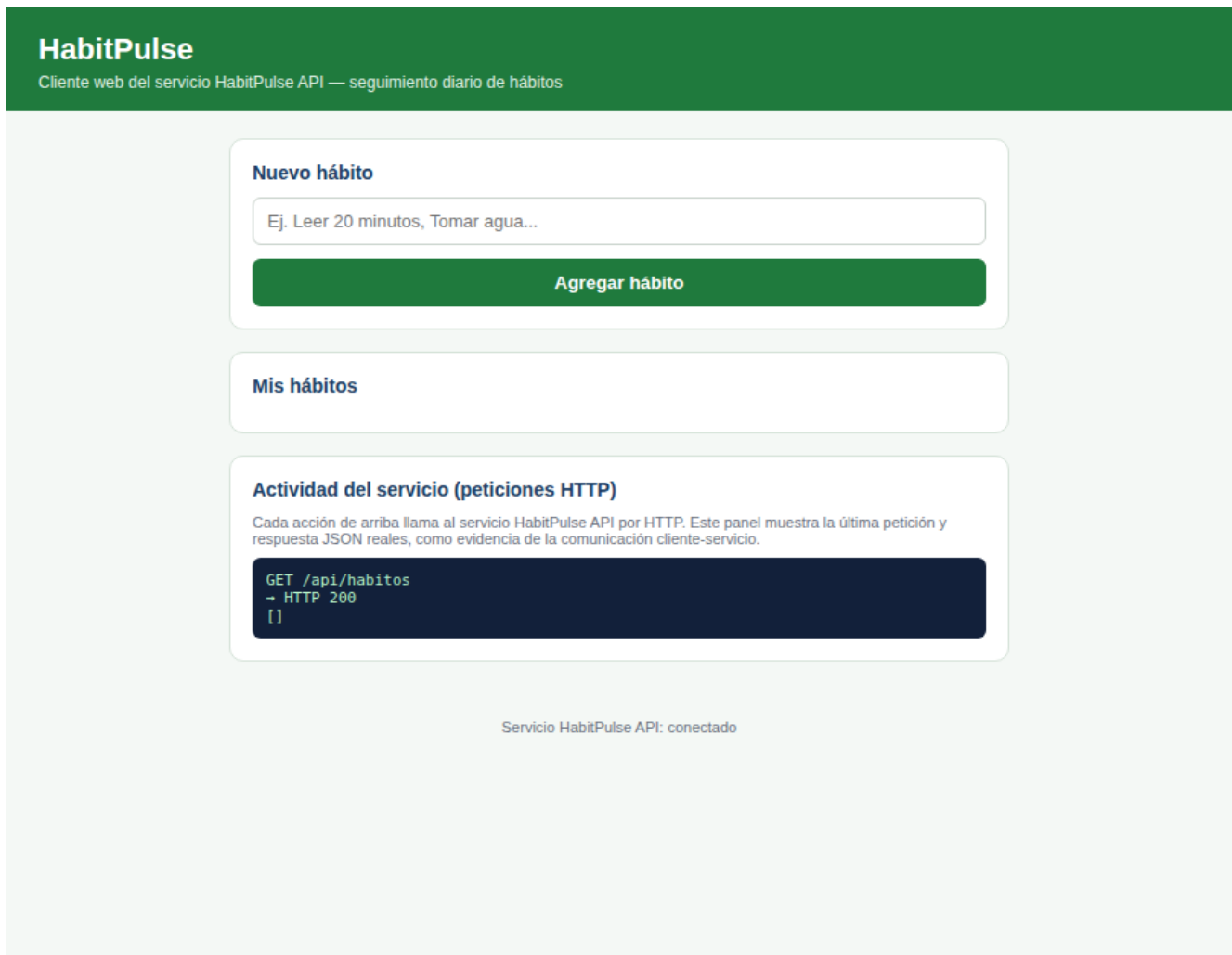


Figura 2. Cliente web de HabitPulse recién cargado: sin hábitos registrados todavía, ya conectado al servicio (footer: "Servicio HabitPulse API: conectado").

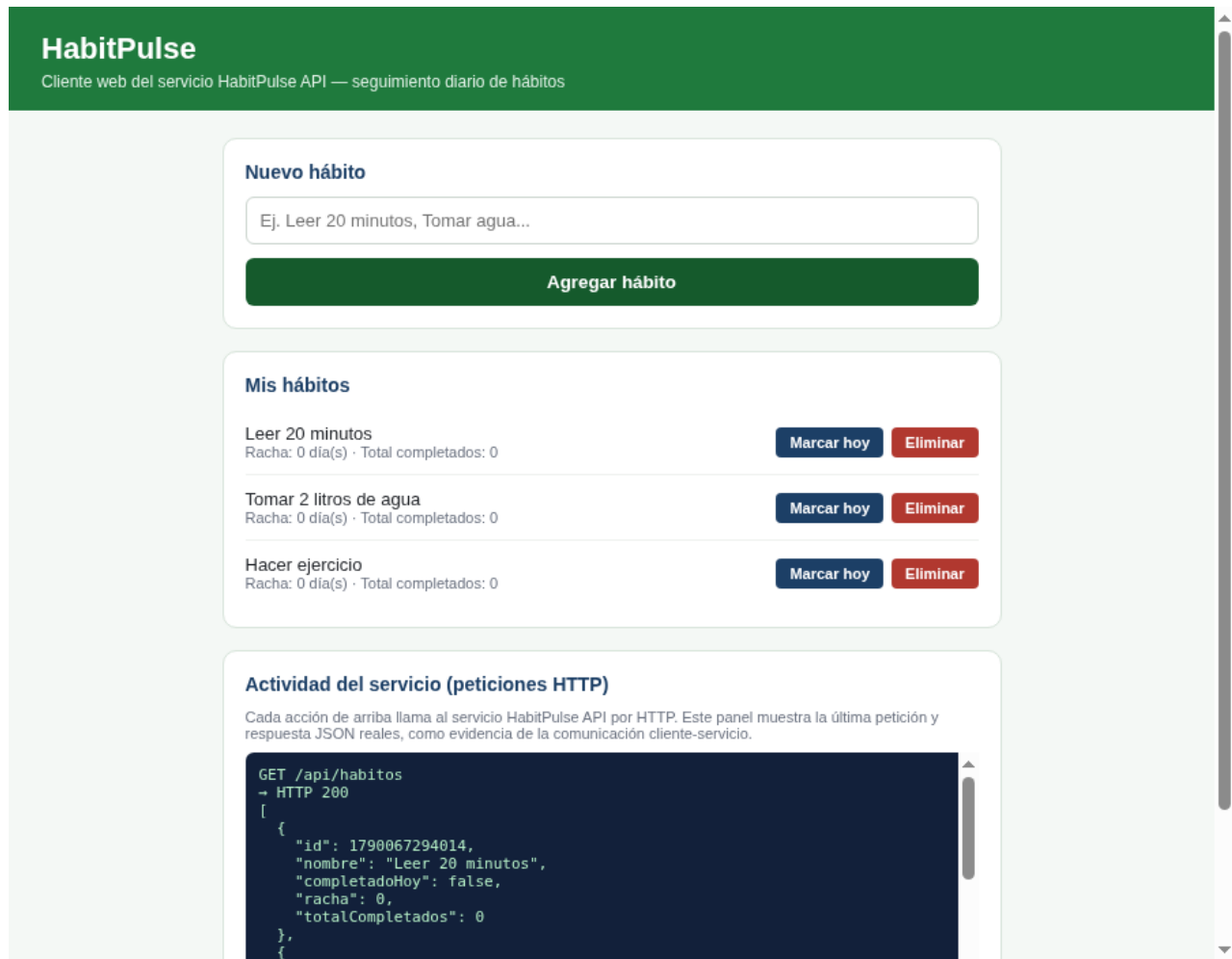


Figura 3. Tres hábitos creados desde el cliente web ("Leer 20 minutos", "Tomar 2 litros de agua", "Hacer ejercicio"), cada uno creado con una petición POST /api/habitos independiente al servicio.

HabitPulse

Cliente web del servicio HabitPulse API — seguimiento diario de hábitos

Nuevo hábito

Ej. Leer 20 minutos, Tomar agua...

Agregar hábito

Mis hábitos

Leer 20 minutos

Racha: 1 día(s) · Total completados: 1

Hecho hoy ✓

Eliminar

Tomar 2 litros de agua

Racha: 0 día(s) · Total completados: 0

Marcar hoy

Eliminar

Hacer ejercicio

Racha: 0 día(s) · Total completados: 0

Marcar hoy

Eliminar

Actividad del servicio (peticiones HTTP)

Cada acción de arriba llama al servicio HabitPulse API por HTTP. Este panel muestra la última petición y respuesta JSON reales, como evidencia de la comunicación cliente-servicio.

```
GET /api/habitos
→ HTTP 200
[
  {
    "id": 1790067294014,
    "nombre": "Leer 20 minutos",
    "completadoHoy": true,
    "racha": 1,
    "totalCompletados": 1
  },
  {

```

Figura 4. Primer hábito marcado como completado hoy: el botón cambia a "Hecho hoy ✓" y la racha pasa a 1 día, tras una petición POST /api/habitos/:id/completar.

HabitPulse

Cliente web del servicio HabitPulse API — seguimiento diario de hábitos

Nuevo hábito

Ej. Leer 20 minutos, Tomar agua...

Agregar hábito

Mis hábitos

Leer 20 minutos

Racha: 1 día(s) · Total completados: 1

Hecho hoy ✓

Eliminar

Tomar 2 litros de agua

Racha: 1 día(s) · Total completados: 1

Hecho hoy ✓

Eliminar

Hacer ejercicio

Racha: 0 día(s) · Total completados: 0

Marcar hoy

Eliminar

Actividad del servicio (peticiones HTTP)

Cada acción de arriba llama al servicio HabitPulse API por HTTP. Este panel muestra la última petición y respuesta JSON reales, como evidencia de la comunicación cliente-servicio.

```
GET /api/habitos
→ HTTP 200
[
  {
    "id": 1790067294014,
    "nombre": "Leer 20 minutos",
    "completadoHoy": true,
    "racha": 1,
    "totalCompletados": 1
  },
  {
```

Figura 5. Panel de actividad del servicio mostrando la última respuesta JSON real: dos hábitos con completadoHoy en true y racha de 1 día, evidencia de que el cliente solo conoce al servicio por HTTP.

HabitPulse

Cliente web del servicio HabitPulse API — seguimiento diario de hábitos

Nuevo hábito

Ej. Leer 20 minutos, Tomar agua...

Agregar hábito

Mis hábitos

Leer 20 minutos
Racha: 1 día(s) · Total completados: 1

Hecho hoy ✓ Eliminar

Tomar 2 litros de agua
Racha: 1 día(s) · Total completados: 1

Hecho hoy ✓ Eliminar

Actividad del servicio (peticiones HTTP)

Cada acción de arriba llama al servicio HabitPulse API por HTTP. Este panel muestra la última petición y respuesta JSON reales, como evidencia de la comunicación cliente-servicio.

```
GET /api/habitos
→ HTTP 200
[
  {
    "id": 1790067294014,
    "nombre": "Leer 20 minutos",
    "completadoHoy": true,
    "racha": 1,
    "totalCompletados": 1
  },
  {
    "id": 1790067294267,
    "nombre": "Tomar 2 litros de agua",
```

Figura 6. Resultado tras eliminar el hábito "Hacer ejercicio" (petición DELETE /api/habitos/:id): quedan los otros dos hábitos y el panel de actividad sigue mostrando el JSON devuelto por el servicio.

```
terminal — bash

$ curl http://localhost:4000/api/habitos
[
  {
    "id": 1790067294014,
    "nombre": "Leer 20 minutos",
    "completadoHoy": true,
    "racha": 1,
    "totalCompletados": 1
  },
  {
    "id": 1790067294267,
    "nombre": "Tomar 2 litros de agua",
    "completadoHoy": true,
    "racha": 1,
    "totalCompletados": 1
  }
]

$ curl -X POST http://localhost:4000/api/habitos -d '{"nombre":"Meditar 10 minutos"}'
{
  "id": 1790067302394,
  "nombre": "Meditar 10 minutos",
  "completadoHoy": false,
  "racha": 0,
  "totalCompletados": 0
}
```

Figura 7. El mismo servicio HabitPulse API consumido directamente desde una terminal con curl, sin pasar por el cliente web: primero un GET a /api/habitos, luego un POST creando un nuevo hábito "Meditar 10 minutos". Evidencia de que el servicio funciona igual sin importar el cliente.

9. Tabla Comparativa: Requisitos de una Aplicación Orientada a Servicios vs. HabitPulse

De la misma forma en que en los subproductos anteriores comparé mis proyectos contra lo que exige cada definición, aquí reviso, criterio por criterio, en qué va HabitPulse respecto a lo que se espera de una aplicación orientada a servicios real:

CRITERIO ORIENTADO A SERVICIOS	QUÉ EXIGE LA DEFINICIÓN	¿CUMPLE?	EVIDENCIA EN HABITPULSE
Servicio independiente con contrato propio	Debe existir un servicio con endpoints definidos, separado de cualquier cliente en particular	Sí	server/index.js expone /api/habitos con métodos GET, POST y DELETE, documentados en el reporte
Comunicación por HTTP/JSON	El servicio y los clientes deben intercambiar datos en un formato estándar sobre HTTP	Sí	Todas las rutas reciben y devuelven JSON; probado tanto desde el cliente web como desde curl
Al menos un cliente que lo consuma solo por red	Debe existir un cliente que use el servicio exclusivamente a través de su API, sin acceso directo a sus datos	Sí	El cliente web (public/app.js) solo usa fetch hacia /api/habitos; nunca lee habits.json directamente
Reusable por más de un tipo de cliente	El contrato debe poder consumirse desde clientes distintos sin cambiar el servicio	Sí	Se demuestra consumiendo el mismo servicio desde el cliente web y desde la terminal con curl, con el servidor sin modificaciones
Independencia de la implementación interna	Cambiar cómo se guardan los datos no debe afectar el contrato expuesto a los clientes	Sí	datos.js aísla el archivo JSON; podría cambiarse por una base de datos sin tocar las rutas ni los clientes
Manejo de errores del servicio	El servicio debe responder con códigos de estado HTTP adecuados ante peticiones inválidas	Sí	Devuelve 400 si falta el nombre del hábito y 404 si el id no existe, con un cuerpo JSON describiendo el error
Despliegue del servicio accesible por red	El servicio debe poder publicarse para que clientes externos lo consuman por internet	Parcial	El servicio corre y se probó localmente en el puerto 4000; el despliegue en un proveedor de hosting queda como siguiente paso

De los siete criterios revisados, HabitPulse cumple con seis de forma completa y uno está parcial: el servicio ya está construido, probado y documentado corriendo localmente y respondiendo correctamente a distintos clientes, pero aún no lo desplegué en un proveedor de hosting con una URL pública en internet, algo que dejo como siguiente paso fuera del alcance de este reporte.

10. Lista de Cotejo

Reviso, a manera de autoevaluación, que el documento cumpla con todos los puntos solicitados para esta actividad:

ELEMENTO SOLICITADO	CUMPLE	OBS.
Portada con formato oficial UTP	Sí	
Índice / tabla de contenido	Sí	
Introducción	Sí	
Definición de qué es una aplicación orientada a servicios	Sí	
Ilustración del funcionamiento (diagrama)	Sí	
Características principales	Sí	
Ventajas y desventajas	Sí	
Comparativa (orientada a servicios vs. móvil vs. web vs. escritorio vs. multiplataforma)	Sí	
Tecnología utilizada	Sí	
Ejemplo de mi propia aplicación orientada a servicios (HabitPulse)	Sí	
Imágenes de la aplicación (evidencia propia)	Sí	Capturas reales del cliente web y de la API por curl
Tabla comparativa: requisitos vs. mi aplicación	Sí	
URL de la aplicación (repositorio)	Sí	
Conclusión	Sí	
Referencias	Sí	

11. Conclusión

Las aplicaciones orientadas a servicios resuelven un problema distinto al de los cinco enfoques anteriores: en vez de definir cómo se construye y distribuye un cliente, definen cómo se organiza la lógica y los datos detrás de él, de modo que puedan reutilizarse desde varios clientes sin duplicar código. Construir HabitPulse como un servicio separado de su cliente web me obligó a pensar en un contrato explícito —qué rutas existen, qué reciben y qué devuelven— en vez de simplemente escribir funciones que la interfaz llama directamente, que es como había resuelto la comunicación en subproductos anteriores.

Demostrar que el mismo servicio responde igual desde el cliente web y desde curl en la terminal hizo tangible la idea central de este enfoque: el servicio no sabe ni le importa quién lo está consumiendo, siempre que la petición respete el contrato HTTP/JSON. Esa independencia es la que permitiría, en un proyecto real, agregar una app móvil o un segundo cliente sin tocar el servicio, o cambiar cómo se guardan los datos sin que ningún cliente existente se entere.

Con este sexto subproducto cierro un panorama completo de seis enfoques de desarrollo de software —nativo (Wear OS), web (MediCore), escritorio (TaskDesk), móvil (PediatraTrack Global), multiplataforma (ExpenseTrack Lite) y orientado a servicios (HabitPulse)— documentados con el mismo tipo de análisis: definición, características, ventajas y desventajas, comparativa y tabla de cumplimiento. Esto me deja claro que estos enfoques no son excluyentes entre sí: una aplicación real con frecuencia combina varios, por ejemplo, un cliente móvil o de escritorio que consume uno o varios servicios propios, como los que documenté por separado a lo largo de esta materia.

12. URL de la Aplicación

Como evidencia del ejemplo presentado (HabitPulse), se listan sus datos de acceso:

- **Repositorio de código fuente:** <https://github.com/AlexAP1115web/HabitPulse.git>

El servicio aún no está desplegado en un proveedor de hosting con una URL pública: se ejecuta y se demuestra corréndolo localmente. PASOS: al clonar el repositorio, dentro de la carpeta del proyecto ejecutar en una terminal con Node.js instalado:

```
npm install
```

```
npm start
```

Esto levanta el servicio HabitPulse API en `http://localhost:4000`, y sirve también el cliente web incluido en la carpeta `public/` en esa misma dirección. El servicio puede consumirse igualmente desde cualquier otro cliente HTTP, por ejemplo:

```
curl http://localhost:4000/api/habitos
```

Referencias

OpenJS Foundation. (2026). *Express documentation*. <https://expressjs.com/>

OpenJS Foundation. (2026). *Node.js documentation*. <https://nodejs.org/en/docs>

Mozilla Developer Network. (2026). *HTTP documentation*.
<https://developer.mozilla.org/es/docs/Web/HTTP>

Mozilla Developer Network. (2026). *Cross-Origin Resource Sharing (CORS)*.
<https://developer.mozilla.org/es/docs/Web/HTTP/CORS>

Fielding, R. (2000). *Architectural styles and the design of network-based software architectures*. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Postman. (2026). *REST API tutorial*. <https://learning.postman.com/docs/introduction/overview/>

cURL. (2026). *cURL documentation*. <https://curl.se/docs/>

Git. (2026). *Git documentation*. <https://git-scm.com/doc>